

INTERVIEW PLAYBOOK / CYPRUS

Full-Stack iGaming Developer

Interview questions, adaptable sample answers, and practical platform use cases

Built from the complete role description

Kotlin backend, modern frontend, APIs, microservices, data, providers, payments, reliability, compliance, delivery, and team collaboration.

A practical preparation guide with 42 interview questions and answers

Important: these are model answers, not claims about your background. Replace every personal detail with something true. The role requires 3+ years of full-stack work, prior iGaming experience, and hands-on Kotlin, so prepare specific evidence for each.

How to use this guide

Read the answers aloud, then rewrite them in your own words. Keep most responses to 60-90 seconds. For technical design questions, use this sequence: requirement, constraints, design, trade-off, failure handling, verification, result.

Make your answers credible

Use exact product, language, framework, and provider names only when you have direct experience. Where you lack a direct example, state that clearly and explain the nearest transferable experience and how you would approach the problem.

A reliable answer structure

1 Context	What product, user, and business problem were involved?
2 Ownership	What did you personally design, build, test, or operate?
3 Trade-off	What alternatives or constraints mattered?
4 Verification	How did you test, release, monitor, and confirm the result?
5 Outcome	What changed for players, operations, reliability, or the business?

A typical platform flow

PLAYER ACTION	PLATFORM CHECKS	INTEGRATION / EVENT	OPERATIONS
Register, deposit, launch, or withdraw	Identity, market eligibility, account status, limits, authorization	Payment or game provider request; callback; wallet ledger; event publication	Monitor, investigate, reconcile, restrict, or support

Interview scope: 42 core questions below cover every responsibility and every required/nice-to-have item named in the posting. The answers focus on practical use, ownership, and trade-offs.

1. Your story and the iGaming business

Open with the business problem and your personal contribution. This role explicitly requires prior iGaming experience, so be exact about what you owned and what you only observed.

Tell me about yourself and the experience most relevant to this role.

SAMPLE ANSWER

A clear answer takes about 60 to 90 seconds: 'I am a full-stack developer with [X years] working on [real products and domains]. My strongest areas are [backend language and frontend stack]. In my iGaming work at [company or product, if true], I owned [specific service or feature] from requirements through production support. One example is [problem], where I [your actions] and the result was [measured outcome]. I work closely with product, QA, design, and operations, and I am looking for a role where I can keep building reliable player-facing and operational systems.' The posting asks for 3+ years, so state your dates and scope clearly. Replace every bracket with a true fact.

HOW THIS IS USED

Use one compact, relevant story: a player or operator problem, your contribution, and a result such as lower failure rate, faster settlement, fewer support tickets, or better conversion. Do not claim an iGaming employer or Kotlin experience you did not have.

Walk me through the real-money gaming platform you worked on and what you personally owned.

SAMPLE ANSWER

Describe the product at a useful level: player account and eligibility, lobby, game launch, wallet or payment flow, provider integration, promotions, support tools, and reporting. Then draw a boundary around your own work: the services, endpoints, screens, decisions, and production duties you personally handled. Explain one interaction across components and how you knew it worked. If your exposure was limited, say which parts you built directly and which you integrated with.

HOW THIS IS USED

For a game launch, you might own the launch API and frontend flow while another team owns the wallet. Explain the session token, provider launch URL, eligibility check, and what your feature does when the provider is unavailable.

How does a real-money gaming platform work from a player's point of view?

SAMPLE ANSWER

A typical journey is registration, identity and age checks, deposit, lobby discovery, game launch, gameplay events, wallet updates, withdrawal, and ongoing account controls. Several services participate: frontend, identity/KYC, wallet or payment, game aggregation/provider, risk controls, CRM, and operations. The user sees one product, but the system must keep eligibility, money movement, game state, and audit records consistent. The precise flow depends on the operator, provider contract, and market.

HOW THIS IS USED

When a player launches a game, the platform can verify account status and jurisdiction, create a short-lived launch session, call the provider, then handle provider callbacks and wallet events. Keep the launch fast, but never bypass a required eligibility or safer-gambling restriction.

Why iGaming, and why a full-stack role that also expects business understanding?

SAMPLE ANSWER

I like work where engineering decisions have a visible effect on a live product. Full-stack ownership helps me understand the whole path from a player action through frontend behavior, APIs, integrations, and operations. In iGaming, reliability and correctness affect both player trust and business results, so I would want to understand the requirement, risk, and success measure before choosing an implementation. Tie this to a real reason and product story from your background.

HOW THIS IS USED

For a deposit issue, the user-facing symptom may be a disabled button, while the actual cause is a payment provider timeout or a delayed webhook. Investigating the whole path avoids a frontend-only patch that hides an unresolved money-state problem.

How do you turn a business request into a technical solution?

SAMPLE ANSWER

I first clarify who needs the change, the player or operator problem, success criteria, edge cases, authorization rules, data changes, and rollout constraints. I write acceptance criteria and agree on examples with product and QA, then map the smallest complete technical path: UI, API, data, integrations, observability, and fallback behavior. I review trade-offs with the team and split the work into testable increments. If a requirement is ambiguous, I state the assumption and get it resolved before it becomes production behavior.

HOW THIS IS USED

For a promotion limit, clarify whether the limit is per player, campaign, currency, or time window; whether pending rewards count; and what an operator should see when the limit is reached. These answers drive constraints and tests.

2. Backend engineering, Kotlin, and system design

The posting names Kotlin as the primary backend language. Expect practical questions about implementation, API behavior, correctness, and the boundaries between services.

What is your hands-on Kotlin experience?

SAMPLE ANSWER

Be direct about the amount and context of your Kotlin work. If it is a strength: describe a service you built, the Kotlin/JVM libraries or framework actually used, how you handled nullability, validation, errors, and tests, and one production result. If you have not used Kotlin professionally, say so plainly and explain the closest backend experience you do have, how Kotlin's type system and JVM ecosystem relate to it, and how you would ramp up. The posting names Java as an advantage and Node.js, PHP, or .NET as examples of other backend experience. State which of these you have actually used. Kotlin is a strong hands-on requirement, so do not imply syntax familiarity equals production experience.

HOW THIS IS USED

A credible example could describe a typed request model, explicit validation, a service boundary, repository call, and integration test for a game-session endpoint. Only name a framework such as Spring or Ktor if you have used it.

How would you structure a Kotlin backend service?

SAMPLE ANSWER

Keep transport concerns, domain rules, and persistence or provider adapters separate. Validate request shape at the API boundary, enforce business invariants in the domain/service layer, and keep database and third-party calls behind clear interfaces. Use Kotlin's non-null types and sealed result or error models to make states explicit, while avoiding abstraction for its own sake. Add structured logs and metrics around meaningful operations. The exact framework should follow the existing platform conventions.

HOW THIS IS USED

A wallet service can expose a transfer command, validate the player and amount, apply a transactional ledger operation, and publish an event after commit. A provider-specific callback parser belongs in an adapter rather than leaking vendor JSON through the domain.

How do you design a good REST API that is both built and consumed by other systems?

SAMPLE ANSWER

Choose resource and action semantics that match the domain, use consistent status codes and error shapes, validate all input on the server, and document required fields, auth, idempotency, pagination, and versioning. For consumers, set timeouts, parse errors safely, validate responses, and treat provider data as untrusted. Define compatibility expectations and contract tests so one service can evolve without silently breaking another.

HOW THIS IS USED

A create-payment endpoint might accept an idempotency key, return a stable payment ID and state, and let the client query that state. The frontend can show pending rather than inventing a success result when the gateway response is delayed.

How do you decide service boundaries in a microservice platform?

SAMPLE ANSWER

Choose boundaries around cohesive business capabilities and ownership of their data, such as identity, wallet, game integration, promotions, or reporting. A service should have a reason to change and scale independently; splitting every table into a service adds network and operational cost. Keep synchronous calls for queries or steps that need an immediate answer, and asynchronous events for work that can happen later. Define ownership, failure behavior, and observability before splitting.

HOW THIS IS USED

The game integration service can own provider credentials and protocol translation, while the wallet owns the authoritative balance and ledger. Neither service should update the other's database directly.

How do you keep money and balances correct under concurrent requests?

SAMPLE ANSWER

Represent monetary values exactly, using integer minor units or a suitable fixed-precision decimal with an explicit currency; never binary floating point. Enforce invariants in the database transaction, use unique idempotency keys or event IDs, and make debit/credit changes atomic. Define ordering and reversal rules, record immutable ledger entries, and reconcile against external statements or provider records. Retries must return the prior result rather than apply the same movement again.

HOW THIS IS USED

If two duplicate win callbacks arrive together, a unique provider transaction key prevents a double credit. If the callback times out after the commit, retrying returns the recorded transaction outcome.

How would you handle distributed transactions across a wallet, game provider, and promotion service?

SAMPLE ANSWER

Avoid pretending that one database transaction can cover several independent systems. Make each step explicit, persist a durable state transition, and use an idempotent workflow with retries and compensating actions where appropriate. An outbox can atomically save a database change and a message-to-publish record. Consumers should tolerate duplicate delivery. For money-critical flows, define a reconciliation path for partial or delayed outcomes instead of relying only on compensations.

HOW THIS IS USED

A bonus grant can be recorded as pending, published to the wallet, marked complete after an acknowledgement, and retried by a worker. An operator view should expose stuck states for safe investigation.

How would you design a game-provider integration from launch to callback?

SAMPLE ANSWER

Place each provider behind an adapter that translates the internal contract to that provider's protocol. On launch, verify the account is eligible, create a short-lived session, and call the provider with a bounded timeout. Verify callback signatures, validate amounts and identifiers, deduplicate transactions, and record the provider reference. Track callback lag and failures; provide retries where safe and reconciliation for missing or disputed events.

HOW THIS IS USED

A player may receive a launch link immediately, while settlement callbacks arrive later. Keep launch-session state separate from wallet transaction state so a provider outage does not make the balance ambiguous.

How do you handle timeouts, retries, circuit breakers, and duplicate requests?

SAMPLE ANSWER

Set timeouts at every network boundary based on the user or workflow budget. Retry only transient failures, with bounded exponential backoff and jitter, and only when the operation is safe to repeat. Use idempotency keys for commands that could create a charge, session, or credit. A circuit breaker can stop repeated calls to a failing dependency and protect the rest of the system. Add a clear pending state when the outcome is unknown.

HOW THIS IS USED

If a payment request times out after submission, do not blindly create another charge. Query by the original merchant reference or reuse its idempotency key, then reconcile the final status.

3. Integrations, data, and performance

This role touches several external vendors and relational data stores. Interviewers are looking for safe boundaries, measurable performance work, and reliable data handling.

How would you integrate game providers, payment gateways, KYC, CRM, affiliate platforms, and operational tools?

SAMPLE ANSWER

Use a clear adapter per external system and a stable internal model, so vendor-specific payloads do not spread across the product. Store credentials in a secret manager, verify signatures, use least-privilege access, and validate input. Add timeouts, safe retries, idempotency where supported, audit records, and monitoring for each integration. Confirm data ownership, retention, and failure workflows with product, security, and operations.

HOW THIS IS USED

KYC can return pending, approved, or rejected rather than a single Boolean. CRM events should be emitted only for the permitted player state. Affiliate attribution needs a defined source of truth and fraud controls.

A provider has an outage or changes its API. What do you do?

SAMPLE ANSWER

First confirm scope and impact using health checks, error rates, provider status, and affected flows. Protect players from repeated failed actions, isolate the dependency with bounded timeouts or a circuit breaker, and keep the UI state truthful. Notify operations through the agreed incident path. Compare the provider contract and logs, reproduce the issue, deploy a compatible adapter fix behind tests, and reconcile any missed transactions afterward. Document the root cause and prevention.

HOW THIS IS USED

For a game provider outage, you might temporarily hide only that provider's games, preserve access to unaffected games, and expose an operator-visible incident status rather than taking down the whole lobby.

Which iGaming providers or platforms have you used?

SAMPLE ANSWER

Answer with vendors you have directly integrated or operated, and describe your role. The posting mentions Smartico, Betby, GR8 Tech, SoftSwiss, EveryMatrix, Pragmatic Play, and Evolution as examples. If you have not used one, say so; then explain how you would assess its API documentation, authentication, sandbox, callback semantics, certification requirements, support process, and operational risks before building an adapter.

HOW THIS IS USED

For a sportsbook feed, casino content provider, or CRM integration, compare launch or event protocols, settlement semantics, sandbox fidelity, support SLAs, and reconciliation tools. Do not present familiarity with the name as integration experience.

How do you design a relational schema for transactional platform data?

SAMPLE ANSWER

Start with the invariants and access patterns. Use primary and foreign keys, unique constraints, appropriate nullability, explicit currency and status fields, and transactions for related updates. Keep immutable financial events or ledger entries where auditability matters. Use migrations that are reviewed, reversible when possible, and safe to roll out while old and new application versions overlap. Avoid storing the same authoritative fact in multiple places.

HOW THIS IS USED

A wallet ledger can use a unique provider transaction reference, player and currency keys, debit/credit direction, exact amount, event time, and source. A current balance can be derived or maintained transactionally with a reconciliation check.

How do you find and fix a slow database query?

SAMPLE ANSWER

Measure first: identify the endpoint and query, inspect latency distribution and volume, then review the query plan and rows scanned. Check indexes against actual filters, joins, and sort order; confirm selectivity and avoid unnecessary columns or N+1 access. Test on representative data, watch write costs and lock behavior, and compare before and after metrics. Add caching only when it fits the freshness requirements.

HOW THIS IS USED

For a slow player transaction history, a composite index on player, currency or status as appropriate, and descending event time may help. Verify with an explain plan rather than adding an index by intuition.

When would you choose NoSQL instead of PostgreSQL, MySQL, or MSSQL?

SAMPLE ANSWER

Choose from access patterns and consistency needs, not fashion. Relational databases are a strong default for transactional records, constraints, and joins. NoSQL may fit high-volume flexible events, key-value lookups, or a specific scaling pattern if the team can operate it and tolerate its consistency model. Define ownership, backup, retention, and migration strategy before introducing another datastore.

HOW THIS IS USED

A cache or session lookup may suit a key-value store, while wallet ledger entries need relational transactions and constraints. Keep one authoritative source for financial state.

How do you approach performance and scalability under high load?

SAMPLE ANSWER

Establish a baseline and service objectives, identify the constrained component, then scale or optimize that component. Use load tests based on realistic traffic and bursts, protect downstream systems with queues or limits, and avoid unbounded work in request handlers. Check database pool sizes, hot keys, payload size, cache behavior, and provider rate limits. Keep correctness under concurrency and graceful degradation part of the load plan.

HOW THIS IS USED

For a large tournament launch, precompute read-heavy lobby data, rate-limit expensive actions, queue non-urgent notifications, and protect the wallet and game provider with independent capacity controls.

4. Frontend, security, and quality

Full-stack means owning the user-visible behavior as well as the API contract. Show that security, accessibility, and failure states are part of the feature.

What modern frontend work have you done with React, Next.js, Angular, or Vue?

SAMPLE ANSWER

Describe the framework and the product surface you actually built. Explain how components, state, API calls, validation, loading, empty, and error states fit together. Mention responsive behavior, accessibility, and performance only where you can give a concrete example. If the role uses a different framework, explain the transferable concepts and how you learn its conventions.

HOW THIS IS USED

A game lobby can render a fast initial catalogue, lazy-load artwork below the fold, preserve filters in the URL, and show a recoverable error if the catalogue API fails.

How should a frontend handle a payment or game launch API error?

SAMPLE ANSWER

Show a truthful, understandable state and a safe next action. Distinguish validation errors, authorization restrictions, transient outages, and unknown outcomes. Do not tell the player a payment succeeded until the server confirms it; for an unknown outcome, show pending and let the player check status. Keep detailed diagnostics out of the UI and send them to structured server-side logs.

HOW THIS IS USED

If a launch call times out after the provider may have created a session, offer a retry that reuses the request identity or ask the backend for the existing launch result.

What security practices do you apply across frontend and backend?

SAMPLE ANSWER

Enforce authorization on the server for every protected action, validate input, protect secrets, use secure transport and session handling, and minimize sensitive data exposure. Apply least privilege, rate limits, audit events for high-risk operations, and safe error responses. Treat provider callbacks and browser data as untrusted. Keep dependencies patched and include security review in design and code review.

HOW THIS IS USED

Never trust a hidden admin button as authorization. The API must independently check the user's role and scope before changing a player's status or issuing a reward.

How do you work with QA to get useful test coverage?

SAMPLE ANSWER

Agree on acceptance criteria and risk areas before implementation. Cover unit tests for domain rules, integration tests for persistence and adapters, contract tests for API/provider boundaries, and end-to-end tests for critical player journeys. Test negative paths, duplicates, timeouts, concurrency, and permission rules. QA should be able to reproduce cases in a stable environment with useful test data.

HOW THIS IS USED

For a deposit, test accepted, declined, pending, duplicate callback, callback after timeout, amount mismatch, and an unauthorized status lookup.

How do you test a microservice integration when the vendor sandbox is unreliable?

SAMPLE ANSWER

Use a combination of contract tests against the documented schema, deterministic stubs for edge cases, and a small number of sandbox smoke tests. Keep fixtures for success, timeout, malformed response, signature failure, and duplicate callback. Avoid tests that pass only because a mock mirrors the implementation; validate key assumptions against the provider contract and real sandbox when available.

HOW THIS IS USED

A callback test fixture can verify signature parsing, idempotency, and event mapping without depending on the provider being online every CI run.

5. Delivery, operations, and team collaboration

The position owns work through release and production support. Explain how you keep changes reviewable and how you learn from incidents.

Describe your software delivery lifecycle from planning through post-release support.

SAMPLE ANSWER

I clarify the requirement and risk, design a small implementation, agree on tests, and develop through reviewed Git changes. CI runs checks and produces a deployable artifact. I coordinate deployment timing and configuration with DevOps and operations, use a staged rollout where appropriate, and watch service and product metrics after release. If an issue appears, I help diagnose it, communicate impact, and follow through on corrective work.

HOW THIS IS USED

For a wallet change, release behind a feature flag, validate in staging with a provider sandbox, canary a small scope if the architecture supports it, then compare error and reconciliation metrics before full rollout.

What is your experience with Docker and CI/CD?

SAMPLE ANSWER

Describe what you have actually built or maintained: Dockerfiles, local compose environments, pipeline steps, artifact handling, environment configuration, deployment strategy, and rollback. Emphasize repeatability and secret hygiene. If your experience is adjacent rather than ownership, say what part you operated and how you collaborated with DevOps.

HOW THIS IS USED

A service image should run with an explicit non-root user where practical, pinned dependencies, health checks, and configuration supplied from the environment or secret manager rather than baked into the image.

How do you use Git on a team?

SAMPLE ANSWER

I keep changes focused, write clear commits or pull request descriptions, rebase or merge according to team policy, and resolve conflicts by understanding both changes. Reviews should catch correctness, security, maintainability, and missing tests without becoming a style argument. I respond to feedback constructively and make the final change easy to verify.

HOW THIS IS USED

For a provider integration, keep the adapter, contract tests, migration, and operational notes in a reviewable change with rollout and rollback steps.

How do you work in Agile or Scrum without losing engineering quality?

SAMPLE ANSWER

Use planning to clarify outcomes and dependencies, break work into increments that can be tested, and surface uncertainty early. Keep standups focused on progress and blockers, and use review or retrospective to improve delivery. Make technical risks visible in the backlog rather than hiding them as unplanned work. Agile should help the team adapt while retaining clear acceptance criteria and quality gates.

HOW THIS IS USED

If a provider certification is a dependency, split internal adapter work, sandbox validation, and production enablement into explicit tasks with owners and exit criteria.

Tell me about a production issue you investigated.

SAMPLE ANSWER

Use a factual STAR story: situation and player/business impact; the signals you checked; how you narrowed the cause; the fix and its risk; how you verified recovery; and what you changed to prevent recurrence. Distinguish correlation from root cause. Do not expose customer data or blame another team. If you have not handled a live incident, use a real staging or support issue and say so.

HOW THIS IS USED

For missing wallet updates, compare provider callback IDs, queue lag, consumer errors, and ledger records. Stop duplicate credits first, restore processing safely, reconcile the time window, and add an alert or idempotency check if that was the gap.

How do you make an architecture decision when there are competing options?

SAMPLE ANSWER

Start with the requirements and constraints: consistency, latency, availability, team ownership, existing platform standards, cost, and expected change. Compare a simple option with more distributed alternatives, list failure modes and operational burden, and document the decision briefly. Prefer the least complex design that meets the constraints, then revisit if evidence changes.

HOW THIS IS USED

For a new reporting view, a read model or asynchronous projection may reduce load on the wallet database; for a balance decision, a stale projection may be unsafe. The business action determines the right trade-off.

How do you handle technical debt and documentation?

SAMPLE ANSWER

Make debt concrete by describing user, reliability, delivery, or operational cost. Prioritize the items that repeatedly slow work or create risk, and include small improvements in feature work where practical. Document system boundaries, important decisions, runbooks, and recovery steps close to the code and keep them current through review.

HOW THIS IS USED

If every new provider requires copying callback logic, extract and test a shared callback pipeline before adding another integration, provided the common abstraction is real and not just hypothetical.

6. iGaming risk, providers, and useful extras

These topics are listed as advantages, but they matter to platform safety. Speak carefully about market-specific rules and distinguish engineering controls from legal interpretation.

How would you support KYC, AML, responsible gambling, and operational risk controls?

SAMPLE ANSWER

Build controls so that eligibility and account restrictions are enforceable at the server boundary and auditable. Keep a clear state model, restrict access to sensitive documents, log high-risk administrative actions, and prevent a stale frontend state from bypassing a restriction. Work with compliance and product experts on the actual market rules; engineering implements agreed policy rather than guessing legal requirements.

HOW THIS IS USED

If an account becomes restricted, re-check eligibility before deposits, withdrawals, or game launches as required by the platform policy, and make the reason and resulting actions visible to authorized operations staff.

What is event-driven architecture, and where would you use a message broker?

SAMPLE ANSWER

A service publishes an event about a fact that has already occurred, and interested consumers process it independently. A broker can buffer bursts, isolate services, and decouple non-immediate work. The trade-offs are eventual consistency, duplicate delivery, ordering limits, schema evolution, and more operational components. Design consumers as idempotent and provide dead-letter handling, replay controls, and observability.

HOW THIS IS USED

After a settled transaction, publish a wallet event for reporting and CRM. The player balance should not depend on a slow CRM consumer.

What cloud or infrastructure experience do you have?

SAMPLE ANSWER

Name the provider and services you have actually operated, such as compute, managed database, object storage, networking, secrets, logs, or monitoring. Explain what you owned and how you handled environments, access, reliability, and cost. If you have not operated the target cloud, connect your deployment and container experience and show how you would learn its platform conventions.

HOW THIS IS USED

Production and staging should have separate credentials and data, and deployment configuration should be observable and repeatable. Coordinate infrastructure changes with DevOps rather than making hidden manual edits.

How would you approach building a product from concept to production launch?

SAMPLE ANSWER

Validate the user problem and success measure, define the smallest safe release, map dependencies and compliance or provider requirements, and prototype uncertain integrations early. Build with product, design, QA, DevOps, and operations involved. Prepare monitoring, support workflows, migration and rollback plans, and release criteria. After launch, measure behavior and reliability and use that evidence to prioritize iteration.

HOW THIS IS USED

For a new game category, confirm provider licensing and market availability, build the catalog and launch flow, test callbacks and wallet behavior, train operations on failure states, then monitor launch success and session errors.

How do you use AI-assisted development tools responsibly?

SAMPLE ANSWER

Use them for bounded tasks such as explaining unfamiliar code, drafting tests, generating boilerplate, or comparing approaches. Treat output as untrusted: verify behavior, run tests, review security implications, and ensure the code fits team conventions. Do not submit secrets, player data, or proprietary material to a tool unless policy permits it. The engineer remains accountable for the final change.

HOW THIS IS USED

An assistant can draft a parser test matrix for provider callbacks, but a developer must verify the provider contract, edge cases, and whether the implementation can duplicate a financial event.

7. Behavioral questions and your questions for them

Keep behavioral answers specific and concise. Be ready to explain the decision, the trade-off, and the result in plain English.

Tell me about a disagreement with product, QA, design, or operations.

SAMPLE ANSWER

Choose a real disagreement and describe the shared goal, not who was wrong. Explain how you surfaced the constraint, compared options with evidence, agreed on acceptance criteria, and followed through. If the decision was not yours, explain how you supported it and what you learned.

HOW THIS IS USED

A request to show a payment as complete immediately may conflict with gateway confirmation semantics. Offer a pending state that preserves a good user experience while keeping the displayed financial state accurate.

How do you prioritize when a production issue competes with planned work?

SAMPLE ANSWER

Assess player impact, money or compliance risk, scope, and available mitigations. Communicate the trade-off with product and operations, stabilize the highest-risk path, and preserve an owner and update cadence for the planned work. Record follow-up work so an incident is not considered finished merely because the symptom disappeared.

HOW THIS IS USED

A duplicate debit risk should outrank a cosmetic lobby improvement. A localized artwork delay may be mitigated separately while core transactions continue.

How do you explain a complex technical issue to a non-technical teammate?

SAMPLE ANSWER

Start with the user or operational impact, explain the cause using familiar terms, state what is known and uncertain, and give the next action and expected update. Avoid unexplained acronyms. Confirm understanding, especially when the decision affects release timing or player communication.

HOW THIS IS USED

Say: 'The provider accepted the game launch, but its confirmation is arriving late. We are keeping the session pending and checking the provider reference before retrying, so we do not create duplicate sessions.'

How do you communicate in English and collaborate across teams?

SAMPLE ANSWER

The role requests Intermediate (B1+) English or higher, so state your actual level honestly. Then give an example of communicating a requirement, technical decision, risk, or incident in writing and in a discussion. Good collaboration means adjusting detail to the audience, confirming shared understanding, and recording decisions so Product, QA, DevOps, Design, and Operations can act on them.

HOW THIS IS USED

For a release with a delayed provider dependency, write a short status update with player impact, current mitigation, owner, next check-in, and what decision is needed. Use plain words and avoid unexplained acronyms.

What would you focus on in your first 30 to 60 days?

SAMPLE ANSWER

Learn the system boundaries, player and operator journeys, release process, incident history, provider contracts, coding conventions, and the team's current priorities. Pair with product, QA, DevOps, and operations. Deliver a small, well-scoped change early, then identify a measurable reliability or workflow improvement with the team.

HOW THIS IS USED

A useful early win could be improving observability around one high-value provider flow or reducing repeated operational investigation with a runbook and a targeted alert.

Rapid review before the interview

Prepare these proof points before you join the call. Use real numbers only; if a measure is confidential, describe the direction or scale without sharing sensitive information.

Experience	Years, product/domain, iGaming responsibilities, and exactly what you owned.
Kotlin	A service or feature, Kotlin/JVM patterns used, tests, production responsibility, and honest depth.
Frontend	One shipped React/Next.js/Angular/Vue feature with API and failure-state details.
Integrations	Which provider categories and vendor names you used directly; one failure you handled.
Data	A schema, transaction, query, or integrity issue you personally improved.
Delivery	Git review, Docker, CI/CD, environments, Agile collaboration, and release role.
Production	One incident, root cause, mitigation, verification, and lasting prevention.
Impact	A measurable player, business, reliability, or operations outcome.

Terms worth explaining clearly

Idempotency	Repeating the same command or callback does not create a second business effect.
Ledger	An auditable record of money movements; a displayed balance should reconcile to it.
Reconciliation	Comparing internal records with provider or payment records and resolving differences.
Outbox	A durable record written with a database transaction so an event can be published reliably later.

Circuit breaker	A guard that temporarily stops calls to a failing dependency so failures do not cascade.
Contract test	A test that verifies the agreed request/response or event shape between systems.
SLO	A measurable reliability target for a service or user journey.
Least privilege	Giving a user or service only the access it needs for its task.

Good questions to ask the employer

- Which platform domains and services would this role own in the first six months?
- How are Kotlin services structured today, and what frameworks and testing conventions does the team use?
- How are wallet and provider transactions made idempotent and reconciled?
- Which game, payment, KYC, CRM, and affiliate integrations are most important for the roadmap?
- How do Product, QA, DevOps, and Operations participate in release and incident response?
- What reliability or delivery problem would you most like the new developer to improve?

Final reminder: the strongest answer is a truthful, specific story that connects a technical decision to a player, business, or operational outcome.